

Data Structures and Algorithms

Unit-1: Background and concept of Data structures

Concepts of Data types:

* Data types: Data may be alphanumeric or numerical

Data-type=permitted data values + Operations.

* Data Structures:

-> The organized collection of data is known as Data structure.

-> Data structure is representation of the logical relationship existing between individual elements of data.

-> In other words, a data structure is a way of organizing all data items that consists not only the elements stored but also their relationship to each other.

-> We can also define it as a mathematical or logical model of particular organization of data items.

-> Data structures affects the design of both structural and functional aspects of a program.

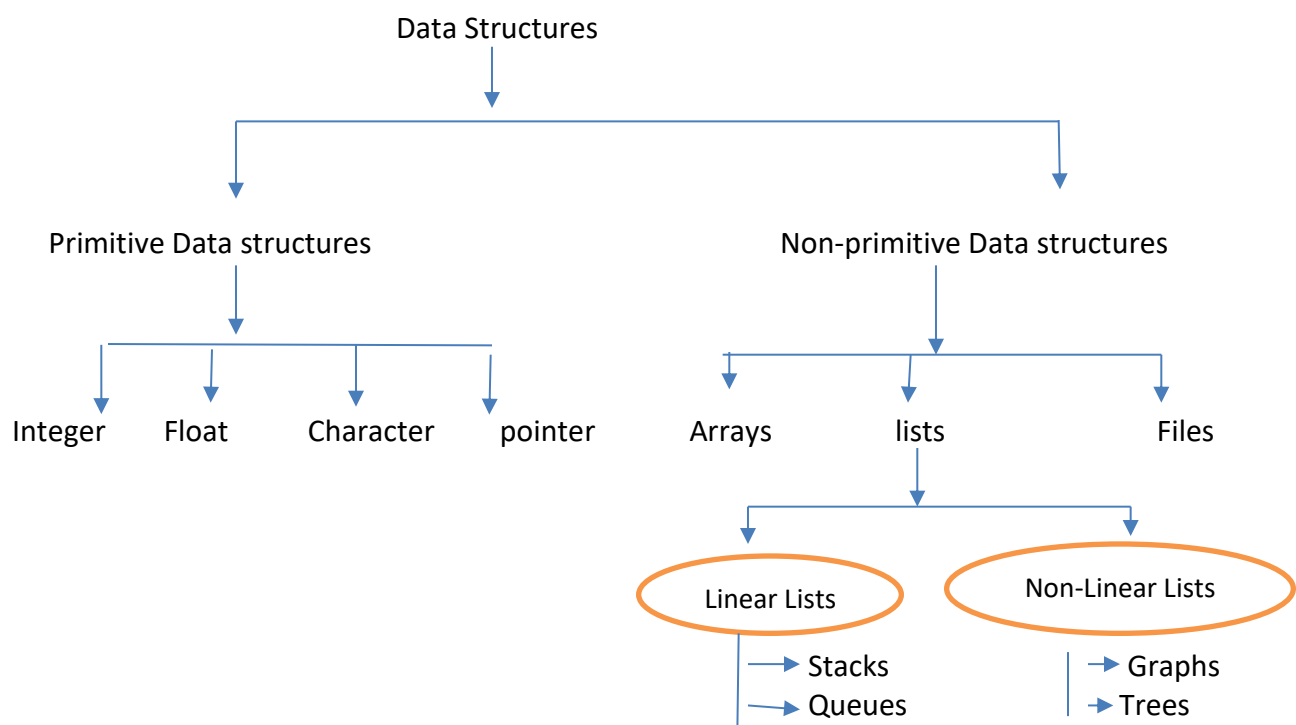
Algorithm + Data structure =program

* Classification of data Structures:

-> Data structures are normally divided into two broad categories:

i. Primitive data Structures

ii. Non-primitive data Structures



* Primitive data Structure:

These are basic data structures and are directly operated upon by the machine instructions. Eg: integer, floating-point numbers, character constants, string constants etc fall in this category.

* Non-primitive data structures:

These are more sophisticated data structures. These are derived from the primitive data structures. The non-primitive data structures emphasize on structuring of a group of homogeneous (same type) or heterogeneous (different types) data items. Eg: arrays, lists, and files.

-> In other words, data structure can be classified into two different types according to three types:

- i. Linear and Non-linear: Which means the data stored in a sequence is called linear, other is non-linear. (Array is linear and Tree is non-linear).
- ii. Homogenous and Non-homogenous: Which means the data structure, which contains single type of data, is known as homogenous and Record is a non-homogenous.

Abstract Data type (ADT):

-> A useful tool for specifying the logical properties of a data type is the abstract data type or ADT.

-> Fundamentally, a data type is a collection of values and a set of operations on those values.

-> The term "abstract data type" refers to the basic mathematical concept that defines the datatypes.

-> To illustrate the concept of an ADT and our specification method, consider the ADT RATIONAL, which corresponds to the mathematical concept of a rational number.

-> The following is an initial specification of this ADT:

```
/* value definition */
abstract typedef<integer,integer>Rational;
Condition RATIONAL[1]!=0;

/*Operator definition*/
abstract RATIONAL makerational(a,b)
int a,b;
precondition b!=0;
postcondition makerational [0]==a;
                makerational [1]==b;

abstract RATIONAL add(a,b) /*written a+b*/
RATIONAL a,b;
postcondition add[1]==a[1]*b[1];
                Add[0]==a[0]*b[1]+b[0]*a[1];
```

```

abstract RATIONAL mult(a,b) /*written a*b */
RATIONAL a,b;
postcondition mult[0]==a[0]*b[0];
                mult[1]==a[1]*b[1];

```

```

abstract RATIONAL equal(a,b) /*written a==b*/
RATIONAL a,b;
postcondition equal==(a[0]*b[1]==b[0]*a[1]);

```

-> An ADT consists of two parts, a value definition and an operator definition.

-> The value definition defines the collection of values for the ADT and consists of two parts: a definition clause and a condition clause.

-> The keywords `abstract typedef` introduce a value definition and keyword `condition` is used to specify any conditions on the newly defined type.

-> Immediately following the value definition comes the operator definition. Each operator is defined as an abstract function with three parts: a header, the optional preconditions and the post conditions.

* Introduction to algorithms:

An algorithm named after ninth century mathematician Al-Khowarizmi, is defined as follows:

-> An algorithm is a set of rules for carrying out calculations either by hand or on a machine.

-> An algorithm is a sequence of computational steps that transform the input into the output.

-> An algorithm is a sequence of operations performed on data that have to be organized in data structures.

* Types of algorithms:

-> **Incremental algorithm:** Insertion sort uses an incremental approach.

Eg:

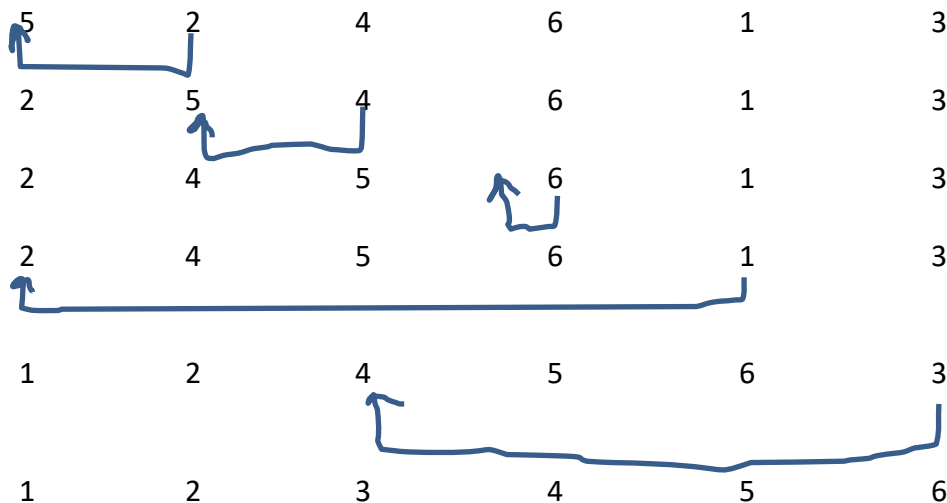


Fig: Insertion Sort

-> Divide and Conquer Algorithm:

- Many useful algorithms are recursive in structures to solve a given problem, they call them solve, recursive one or more time to deal with closely related sub-problems. These algorithms follow a divide and conquer approach.

Divide: the problem into a number of sub-problems

Conquer: the sub-problem by solving them recursively.

Combine: the solutions to the sub-problem into the solution for the original problem(or one).

A={ 5,2,4,6,1,3,2,6}

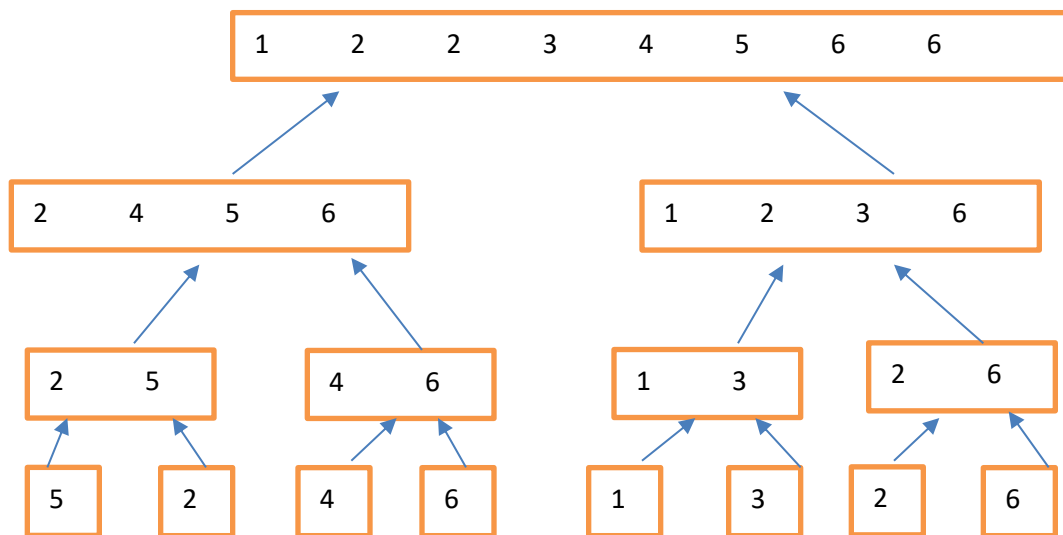


fig: merge sort

-> **Branch and bound algorithm:**

-> used in tree traversing

-> implemented using queue

-> **Backtracking Algorithm:**

-> used in tree traversing

-> implemented using stack.

-> **Randomized Algorithm:**

- used in quick sort